Talaouglid

DOSSIER TECHNIQUE ET FONCTIONNEL

Plateforme de transport de bagages entre pays

Description complète du site : ce que fait chaque partie, qui fait quoi, comment les données circulent, comment le site est hébergé et mis à jour.

Site en ligne	https://talaouglid.onrender.com
Code source	https://github.com/achour2/transport-am (branche `main`)
Technologies	Python 3.12 · Django 5.2 · PostgreSQL · Bootstrap 5 · Leaflet · HTMX
Hébergement	Render (plan gratuit), région Oregon (US West)
Version du document	3 novembre 2026 — commit `f92ce2f` (plan de lancement intégré)
Auteur	Achour Messaoudi

Sommaire

1. Présentation du projet
2. Les acteurs et leurs droits
3. Architecture et technologies
4. Organisation des fichiers : qui fait quoi
5. Base de données : les tables
6. Règles de gestion
7. Les pages du site (adresses et rôles)
8. Parcours complets (de A à Z)
9. Notifications et courriels
10. Commandes, tâches planifiées et API
11. Interface : design, langues, accessibilité
12. Sécurité et confidentialité
13. Tests automatiques
14. Hébergement et mise en ligne (Render)
15. Travailler sur le projet au quotidien
16. Limites actuelles et prochaines étapes
17. Historique des versions
18. Aide-mémoire

> **À qui s'adresse ce document ?** Au gérant (pour comprendre ce que le site sait faire), à un développeur qui reprend le projet (pour savoir où se trouve chaque chose) et à toute personne qui doit le mettre à jour ou le dépanner.

1. Présentation du projet

Le besoin

L'entreprise transporte des bagages et des colis entre le Canada et plusieurs pays (Algérie, France, Maroc, Tunisie). Avant le site, les réservations, les dates de départ et le suivi se faisaient par téléphone ou par messages, ce qui multipliait les erreurs et les appels de clients voulant savoir où était leur bagage.

Ce que fait le site

Pour le public

- Voir les destinations, les tarifs et les prochains départs.
- Calculer un prix exact (simulateur).
- Suivre un bagage avec son numéro (carte du trajet, historique).
- Poser une question (FAQ, formulaire de contact).

Pour le client

- Réserver un envoi en 4 étapes (déclaration de contenu horodatée) ; ses kilos sont bloqués sur le voyage.
- Payer au point de dépôt (comptoir).
- Suivre ses envois, recevoir ses notifications, télécharger son reçu PDF.
- Annuler, gérer son profil, supprimer son compte (Loi 25).

Pour le transporteur

- Voir ses voyages sur téléphone.
- Marquer les bagages « chargés », démarrer le voyage, envoyer sa position GPS en 1 clic.
- Remettre un bagage contre le code à 6 chiffres + photo obligatoire (blocage après 5 codes faux).
- Signaler un incident.

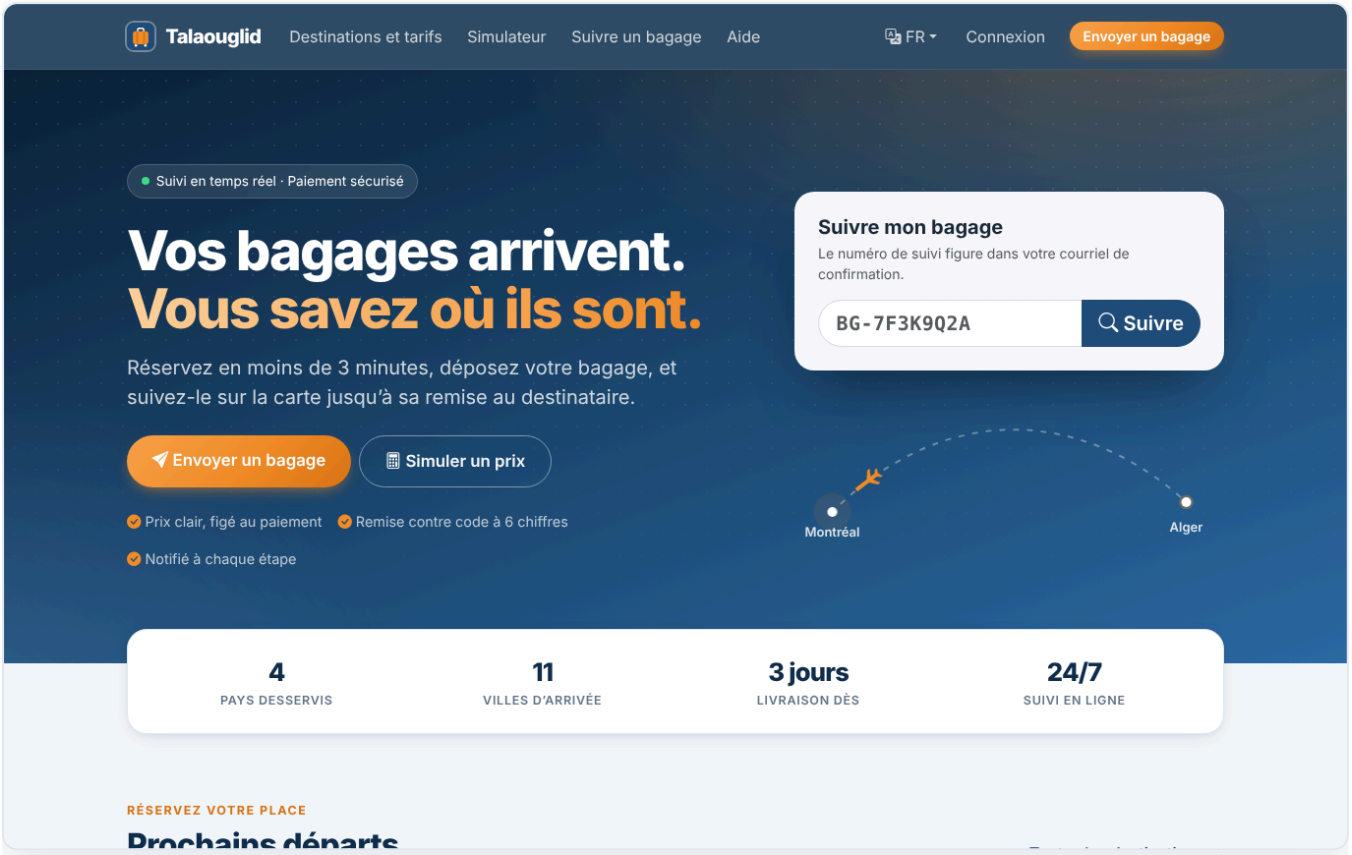
Pour l'administrateur

- Tableau de bord : chiffres, alertes, carte des voyages en cours.
- Gérer pays, tarifs, villes, voyages (calendrier), bagages, incidents, utilisateurs, paramètres.
- Peser et encaisser au comptoir (espèces, terminal, virement), imprimer l'étiquette QR.
- Modérer les avis clients, créer des comptes d'équipe.
- Exporter en Excel.

État actuel (3 novembre 2026)

Élément	État
Site en ligne	En ligne https://talaouglid.onrender.com
Paiement	Au comptoir Le paiement par carte en ligne est désactivé tant que Stripe n'est pas configuré. Le client réserve en ligne et paie au dépôt.

Courriels	À brancher Prêt : il suffit d'ajouter les identifiants Brevo sur Render (guide des actions, étape 4).
SMS / WhatsApp	Version 2 Les messages sont seulement enregistrés dans le journal.
Rappels automatiques	À brancher Adresse `/taches/verifications/` prête (jeton secret) : à appeler toutes les 10 min avec cron-job.org.
Sécurité	Actif Double authentification des admins, blocage après 5 échecs de connexion, piège à robots, limite de requêtes, position publique floutée.
Photos	À brancher Stockage Cloudflare R2 / S3 prêt (variables `S3_*`).
Langues	Français. Anglais et arabe **masqués** tant que la traduction n'est pas complète (`DJANGO_LANGUES`).
Tests	53 tests réussis, lancés aussi automatiquement par GitHub Actions à chaque envoi.



Page d'accueil (ordinateur)

2. Les acteurs et leurs droits

Chaque compte a un **rôle** (champ `role` de la table des utilisateurs). Chaque page vérifie le rôle avant d'afficher quoi que ce soit (fichier `core/decorators.py`) : un client qui tape l'adresse d'une page d'administration reçoit « Accès refusé » (erreur 403).

Acteur	Comment il obtient un compte	Ce qu'il peut faire	Espace
Visiteur	Pas de compte	Voir destinations et tarifs, simuler un prix, suivre un bagage avec son numéro, lire la FAQ, envoyer un message, créer un compte.	Pages publiques
Client	S'inscrit lui-même, ou est créé par l'admin lors d'un envoi au comptoir (lien d'activation par courriel)	Réserver un envoi, voir ses envois et notifications, télécharger ses reçus, annuler avant la date limite, choisir un autre voyage si le sien est annulé, gérer ses destinataires et son profil, supprimer son compte.	`/client/`
Transporteur	Créé par l'admin (Gestion → Utilisateurs)	Voir les voyages qui lui sont assignés, charger les bagages, démarrer / terminer un voyage, envoyer sa position, remettre un bagage avec le code, signaler un incident.	`/transporteur/`
Administrateur	Variables `DJANGO_ADMIN_EMAIL` / `DJANGO_ADMIN_PASSWORD` sur Render, ou `createsuperuser` sur le PC	Tout gérer : pays, tarifs, villes, voyages, bagages, encaissements, remboursements, incidents, utilisateurs (bloquer, réinitialiser), messages, paramètres, export Excel.	`/gestion/`

> **Après connexion**, chacun est redirigé automatiquement vers son espace (page `/mon-espace/`). Le menu du haut change aussi selon le rôle.

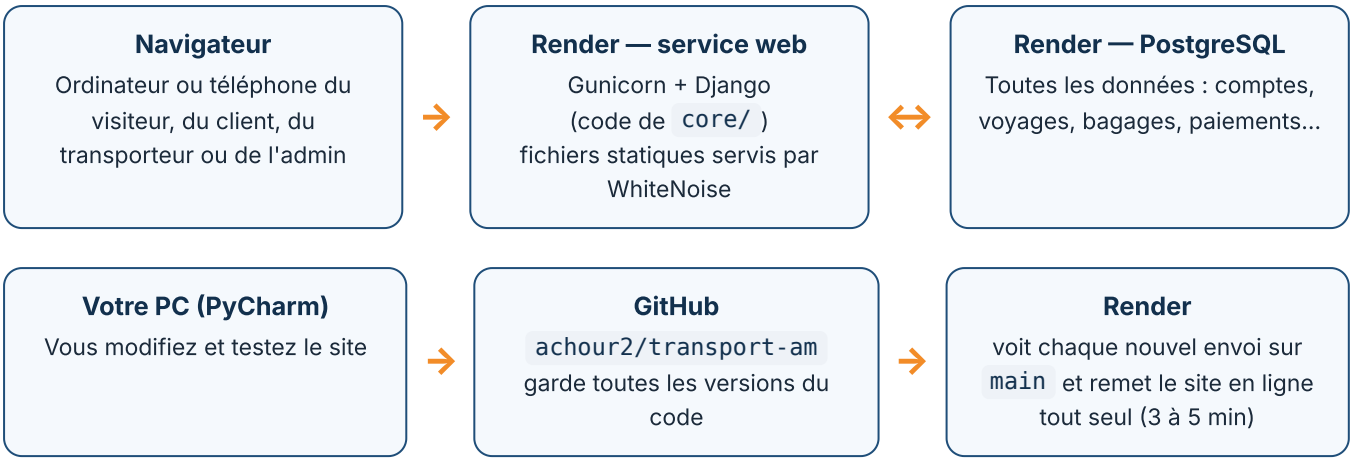
Comptes de démonstration

La commande `donnees_demo` crée des comptes d'essai en `@demo.ca` (administrateur, transporteurs, clients) avec un mot de passe commun. Ils se suppriment avec `supprimer_demo` (section 10). **Ils ne doivent jamais rester actifs en production.**

Administrateurs : double authentification obligatoire (application Google ou Microsoft Authenticator) à chaque connexion. Un admin peut créer d'autres admins et des transporteurs dans Gestion → Utilisateurs.

3. Architecture et technologies

Vue d'ensemble



Technologies utilisées et pourquoi

Besoin	Technologie	Rôle dans le projet
Langage et framework	Python 3.12, Django 5.2	Pages, comptes, sécurité, accès à la base (ORM), administration.
Base de données	PostgreSQL (en ligne), SQLite (sur le PC)	Choisie automatiquement : `DATABASE_URL` en ligne, sinon fichier `db.sqlite3`.
API JSON	Django REST Framework	Données de la carte et d'une future application mobile.
Interface	Gabarits Django + Bootstrap 5.3 + Bootstrap Icons	Pages rapides et adaptées au téléphone ; version droite-à-gauche pour l'arabe.
Calcul instantané	HTMX 1.9	Le simulateur se met à jour pendant la saisie, sans recharger la page.
Carte	Leaflet 1.9 + OpenStreetMap	Trajet, positions du transporteur, carte des voyages en cours. Gratuit, sans clé.
Drapeaux	flag-icons (images SVG)	Remplacent les drapeaux emoji, que Windows n'affiche pas.
Position GPS	Géolocalisation du navigateur	Le téléphone du transporteur envoie ses coordonnées en 1 clic.
Reçus PDF	ReportLab	Génère le reçu de chaque paiement.
Export Excel	openpyxl	Export des envois du mois depuis le tableau de bord.
Paiement en ligne	Stripe (prêt, désactivé)	S'active en ajoutant `STRIPE_SECRET_KEY`.
Courriels	SMTP (Brevo ou SendGrid)	S'active avec `EMAIL_HOST` ; sinon les courriels sont affichés dans le journal.

Tâches planifiées	cron-job.org → `/taches/verifications/`	Rappels, alertes, expiration des réservations, demandes d'avis (toutes les 10 min, jeton secret).
Sécurité	django-axes, django-otp	Blocage après 5 échecs de connexion ; double authentification des admins.
Photos	django-storages (Cloudflare R2 / S3)	Photos conservées hors de Render, liens privés temporaires.
Alertes d'erreurs	Sentry (optionnel)	Courriel à chaque erreur du site, sans données personnelles.
Polices	Inter, Noto Sans Arabic (copiées)	Plus aucun appel à Google Fonts.
Tests automatiques	GitHub Actions + PostgreSQL	Les tests tournent à chaque envoi sur GitHub.
Serveur web	Gunicorn + WhiteNoise	Gunicorn exécute Django ; WhiteNoise sert les CSS, JS et images, compressés et mis en cache.
Hébergement	Render (Blueprint `render.yaml`)	Site + base, mise en ligne automatique depuis GitHub, HTTPS inclus.

> Toutes les bibliothèques de l'interface (Bootstrap, icônes, HTMX, Leaflet, drapeaux) sont **copiées dans le projet** (`static/vendor/`) : le site ne dépend d'aucun service externe pour s'afficher, sauf les fonds de carte OpenStreetMap.

4. Organisation des fichiers : qui fait quoi

Le projet contient environ 3 800 lignes de Python et 53 gabarits HTML. Tout le code « métier » est dans l'application `core`.

Racine du projet

Fichier / dossier	Rôle
`manage.py`	Point d'entrée des commandes Django (`runserver` , `migrate` , `test` …).
`requirements.txt`	Liste des paquets Python à installer.
`render.yaml`	Description de l'hébergement Render : le site, la base, la région, les variables.
`build.sh`	Construction sur Render : installe les paquets et prépare les fichiers statiques.
`start.sh`	Démarrage sur Render : attend la base (2 min max), applique les migrations, crée l'admin, gère la démo, lance Gunicorn.
`.env.example`	Modèle des réglages secrets pour un serveur classique (à copier en `.env` , jamais envoyé sur GitHub).
`.gitignore`	Exclut de GitHub : `.venv` , `db.sqlite3` , `.env` , `media/` , `staticfiles/` .
`README.md`	Démarrage rapide, comptes de démo, mise en ligne.
`deploiement/`	Configuration Nginx et service systemd (VPS), sauvegarde nocturne (serveur) et `sauvegarde-pc.ps1` (copie de la base en ligne sur votre PC).
`.github/workflows/tests.yml`	Tests automatiques sur GitHub à chaque envoi.
`scripts/traductions.py`	Génère les fichiers de traduction anglais et arabe (`locale/`).
`captures/`	Captures d'écran du site.

Dossier `config/` — réglages du projet

Fichier	Rôle
`settings.py`	Tous les réglages : base de données, langues, fuseau horaire, courriels, Stripe, tâches planifiées, sécurité HTTPS, double authentification, blocage des connexions, stockage R2, Sentry, adresse Render, WhiteNoise, paiement en ligne activé ou non (`PAIEMENT_EN_LIGNE`).
`urls.py`	Adresses principales : `/django-admin/` , `/i18n/` (changement de langue), `/api/` , `/favicon.ico` , puis toutes les pages de `core` .
`wsgi.py` / `asgi.py`	Points d'entrée pour le serveur web.

Dossier `core/` — le cœur du site

Fichier	Rôle
`models.py`	Les tables de la base (section 5) et leurs calculs : prix, kilos restants, voyage complet, villes desservies, drapeau, statut affiché…
`services.py`	**Toutes les règles de gestion** (section 6) : réserver, encaisser, confirmer un paiement, rembourser, annuler, charger, démarrer, terminer, livrer, incidents, notifications, rappels périodiques. Toute modification d'un envoi passe par ici, ce qui garantit l'historique et les notifications.
`forms.py`	Les formulaires et leurs contrôles : inscription, profil, simulateur, 4 étapes de réservation, position, incident, livraison, pays, voyage, envoi au comptoir, paramètres…
`views/public.py`	Pages publiques : accueil, destinations, simulateur, suivi, aide, conditions, connexion, inscription, confirmation de courriel, notifications, `robots.txt`, `sitemap.xml`.
`views/client.py`	Espace client : tableau de bord, réservation en 4 étapes, paiement, détail d'un envoi, reçu PDF, annulation, choix après annulation d'un voyage, profil, suppression de compte, webhook Stripe.
`views/transporteur.py`	Espace transporteur : mes voyages, détail d'un voyage, position, remise au destinataire.
`views/gestion.py`	Espace administrateur : tableau de bord, export Excel, pays, voyages et calendrier, bagages, envoi au comptoir, encaissement, reçu, incidents, utilisateurs, messages, paramètres.
`urls.py` / `api_urls.py`	Liste des adresses du site et de l'API (section 7).
`api.py`	API JSON : suivi d'un bagage, pays desservis, voyages en cours (pour la carte de l'admin).
`decorators.py`	Contrôle d'accès par rôle (`client_requis`, `transporteur_requis`, `admin_requis`).
`pdf.py`	Reçu PDF (prix, poids facturé, mode de paiement, NEQ et taxes, lien des conditions) et étiquette 100 × 150 mm avec code QR.
`context_processors.py`	Données disponibles sur toutes les pages : paramètres de l'entreprise, notifications non lues, adresse du site, paiement en ligne activé.
`templatetags/outils.py`	Petits outils d'affichage : couleur d'un statut, date dans le fuseau d'un pays, « il y a 2 h », montant formaté (« 1 234,50 $ »).
`securite.py`	IP réelle derrière Render, limitation de requêtes (anti-robots), obligation de double authentification pour les admins.
`views/deux_facteurs.py`	Page de configuration et de saisie du code à 6 chiffres des admins.
`admin.py`	Interface d'administration Django de secours (`/django-admin/`).
`management/commands/`	Commandes : `donnees_demo`, `supprimer_demo`, `creer_admin`, `verifications` (section 10).

`migrations/`	Historique des changements de la base : 0001 tables initiales, 0002 paiement au comptoir, 0003 nom Talaouglid, 0004 réservation à payer au comptoir, 0005 voyages vers plusieurs villes, 0006 plan de lancement (pesée, volumétrique, sécurité, légal, avis), 0007 villes sans doublons (fusion automatique).
`tests.py`	Les 53 tests automatiques (section 13).

Dossiers `templates/` et `static/` — l'affichage

Dossier / fichier	Rôle
`templates/base.html`	Squelette commun : menu, langue, notifications, messages flottants, pied de page, icônes, référencement Google.
`templates/public/`	Accueil, destinations, simulateur (+ résultat), suivi (recherche et détail), aide, conditions, notifications.
`templates/comptes/`	Connexion, inscription, mot de passe oublié / nouveau / changé, texte du courriel de réinitialisation.
`templates/client/`	Tableau de bord, nouvel envoi (4 étapes), paiement, détail d'un envoi, profil.
`templates/transporteur/`	Mes voyages, détail d'un voyage, position, remise ; bouton GPS réutilisable.
`templates/gestion/`	Toutes les pages de l'administrateur + menu latéral, pagination, champ de formulaire.
`templates/partials/`	Morceaux réutilisés : carte Leaflet, barre de progression du bagage, historique daté, formulaire générique.
`static/css/site.css`	Tout le design : couleurs, boutons, cartes, en-têtes, animations, version téléphone, impression.
`static/js/site.js`	Interactions : menu qui s'affine au défilement, lien actif, messages qui disparaissent, apparition au défilement, indicateur de chargement sur les boutons, compteurs animés.
`static/img/`	Logo (valise) et icônes de l'onglet et du téléphone.
`static/vendor/`	Bibliothèques copiées : Bootstrap, Bootstrap Icons, HTMX, Leaflet, drapeaux.
`locale/en`, `locale/ar`	Traductions anglaise et arabe.

5. Base de données : les tables

Chaque bloc correspond à une table (défini dans `core/models.py`). Les dates sont enregistrées en temps universel (UTC) et affichées dans le fuseau du pays concerné.

Utilisateur

Champ	Contenu
courriel (identifiant), mot de passe	Le courriel sert à se connecter ; le mot de passe est chiffré.
rôle	client, transporteur ou administrateur.
prénom, nom, téléphone, pays de résidence, langue	Coordonnées et langue préférée (fr, en, ar).
notif. site / courriel / SMS	Préférences de notification.
courriel confirmé, actif	Confirmation par lien ; « actif » à faux = compte bloqué.

Table liée : **Destinataire enregistré** (nom, téléphone, adresse, courriel) — carnet d'adresses du client.

Pays et Ville

Champ	Contenu
nom, code (ex. DZ), devise	Le code sert au drapeau.
frais fixes, tarif par kilo, poids maximum	Base du calcul du prix.
délai minimum / maximum (jours)	Affiché « 5 à 7 jours ».
objets interdits, documents de douane	Un élément par ligne.
fuseau horaire, actif	Ex. `Africa/Algiers` ; un pays inactif disparaît des réservations.
Ville : pays, nom, latitude, longitude	Les coordonnées placent la ville sur la carte. Un nom normalisé interdit les doublons (« tizi ouzou » = « Tizi-Ouzou »).

Voyage

Champ	Contenu
ville de départ	Ex. Montréal.
ville d'arrivée (principale)	Ex. Paris.
autres villes desservies	0 à N villes du même pays (ex. Lyon, Marseille). Un voyage dessert donc 1 à N villes.

départ, arrivée prévue, date limite de dépôt	La date limite vaut par défaut 24 h avant le départ.
lieu et heure de dépôt	Où et quand déposer le bagage.
transporteur, moyen (avion, camion, bateau)	Le transporteur assigné est notifié.
capacité (kg)	Les kilos restants sont calculés (section 6).
statut	planifié → en cours → terminé, ou annulé.
dates techniques	Démarré le, terminé le, dernier rappel de position, dernière alerte admin, alerte « presque complet » envoyée.

Table liée : **Position** (latitude, longitude, nom du lieu, commentaire, date, transporteur) — chaque point du trajet.

Bagage (envoi)

Champ	Contenu
numéro de suivi	Unique et non devinable, ex. `BG-7F3K9Q2A` (sans les lettres ambiguës O/0, I/1).
client, voyage, **ville de livraison**	La ville de livraison fait partie des villes desservies par le voyage.
description, poids, dimensions, valeur déclarée	Contenu du bagage.
photo au dépôt, photo à la livraison	Preuves en image.
destinataire : nom, téléphone, adresse, courriel	Le nom est masqué sur la page publique (ex. « S*** B. »).
code de remise	6 chiffres aléatoires, exigés à la livraison.
frais fixes, tarif/kg, prix, devise, poids facturé	**Figés** à la réservation (un changement de tarif ne touche pas cet envoi). Poids facturé = max(poids réel, volumétrique). Recalculés seulement à la pesée au comptoir.
déclaration de contenu, essais de code, remise bloquée, 4 chiffres de la pièce	Preuves et sécurité de la remise.
statut	enregistré, déposé, chargé, en transit, arrivé, livré, incident, perdu, annulé.
statut du paiement	en attente, payé, échoué, remboursé.
paiement au comptoir	Vrai = réservé en ligne, à payer au dépôt (kilos bloqués).
choix requis	Vrai = voyage annulé, le client doit choisir (autre voyage ou remboursement).

Autres tables

Table	Contenu

Événement (historique)	Chaque changement d'un bagage : statut, message, **auteur**, date et heure (traçabilité).
Paiement	Bagage, montant, devise, **mode** (carte, espèces, terminal, virement), encaissé par, référence, statut, dates de paiement et de remboursement. Numéro de reçu : `R-AAAAMM-00012`.
Incident	Voyage, bagage (facultatif), type (retard, douane, endommagé, introuvable, autre), commentaire, état (ouvert, retrouvé, perdu, résolu), auteur.
Notification	Destinataire, message, lien vers la page concernée, lue ou non.
Abonnement au suivi	Courriel d'une personne sans compte qui veut être prévenue (ex. le destinataire).
Message de contact	Messages du formulaire d'aide (traité ou non).
Avis client	Avis demandés 24 h après la livraison, publiés sur l'accueil après modération.
Paramètres (une seule ligne)	Nom de l'entreprise, téléphone, courriel, adresse du dépôt, horaires, fréquence du rappel de position (2 h), délai d'alerte (4 h), fermeture des réservations (24 h), seuil « presque complet » (90 %), diviseur volumétrique, plafond d'indemnisation, nom légal, NEQ, adresse légale, numéros TPS/TVQ, responsable vie privée, textes du courriel, conditions, politique de confidentialité.

6. Règles de gestion

Elles sont toutes codées dans `core/services.py` et vérifiées par les tests.

Prix

> **Prix = frais fixes du pays + (poids en kg × tarif par kilo du pays).** Exemple : 18 kg vers l'Algérie = 20 $ + 18 × 8 $ = **164 $**. Au-delà du poids maximum du pays, la réservation est refusée. Le prix est **figé** au moment de la réservation et dépend du pays, pas de la ville. **Poids volumétrique** (si activé dans Paramètres) : poids facturé = max(poids réel, L × l × H ÷ diviseur). **Pesée au comptoir** : si la balance diffère, le prix est recalculé aux tarifs figés, la capacité revérifiée, et l'historique le note.

Capacité et réservation

- **Kilos réservés** d'un voyage = kilos des envois payés + kilos des réservations « à payer au comptoir » tant que la date limite de dépôt n'est pas passée. Les envois annulés ne comptent pas.
- On ne peut pas réserver plus que les **kilos restants**. À 0 kg restant, le voyage s'affiche « Complet ».
- Les réservations ferment à la **date limite de dépôt** (par défaut 24 h avant le départ, réglable dans Paramètres).
- À l'encaissement, la capacité est revérifiée. Le bagage n'est pas compté deux fois, et deux encaissements simultanés ne peuvent pas dépasser la capacité (verrou en base).

Paiement au comptoir (mode actuel)

1. Le client réserve en ligne : ses kilos sont **bloqués**, il reçoit un numéro et le montant à payer.
2. Au dépôt, l'admin ouvre la réservation (filtre « À encaisser ») et clique sur **Encaisser** (espèces, terminal ou virement), en indiquant si le bagage est déposé.
3. L'envoi passe à « payé » : confirmation, code de remise et reçu deviennent disponibles.
4. Sans paiement avant la date limite, les kilos ne sont plus bloqués. La réservation est annulée automatiquement par les vérifications périodiques ; l'admin peut aussi l'encaisser tard s'il reste de la place, ou l'annuler.

L'admin peut aussi créer directement un envoi pour un client venu sans réservation (**Envoi au comptoir**). Si un courriel est donné, un compte est créé ou retrouvé, avec un lien d'activation ; sinon le client suit son bagage avec le numéro imprimé sur le reçu.

Voyages, pays et villes

- L'arrivée doit être après le départ ; la date limite de dépôt avant le départ.
- Les pays de départ et d'arrivée doivent être **différents et actifs**.
- Les « autres villes desservies » doivent être dans le **même pays** que la ville principale.
- La capacité ne peut pas descendre sous les kilos déjà réservés.
- Un pays utilisé par des voyages **n'est pas supprimé mais désactivé** : il disparaît des réservations, l'historique reste.
- Modifier les dates d'un voyage notifie tous ses clients et le transporteur.

Annulations et remboursements

Situation	Règle
Client, réservation non payée	Annulation libre ; rien à rembourser ; kilos libérés.
Client, envoi payé	Possible jusqu'à la date limite de dépôt et tant que le bagage n'est pas déposé ; remboursement intégral.
Après le dépôt	Annulation impossible en ligne.
L'admin annule un voyage	Chaque client est notifié et choisit : un autre voyage vers le même pays (même prix) ou le remboursement.
Remboursement d'un paiement en espèces	Pas de Stripe : l'admin est averti qu'il doit rendre l'argent en main propre.

Transport, livraison et incidents

- Le transporteur marque chaque bagage **chargé** (bagage payé et non annulé uniquement), puis **démarre** le voyage : les bagages chargés passent « en transit » et les clients sont notifiés.
- Pendant le voyage, il envoie sa position. Sans position depuis **2 h**, il reçoit un rappel ; depuis **4 h**, l'admin reçoit une alerte (une seule fois par période).
- Il marque le voyage **arrivé** : les bagages passent « arrivé à destination ».
- Un bagage passe **livré** seulement avec le **code à 6 chiffres** du destinataire et une **photo de remise** (4 chiffres de la pièce d'identité en option). Après **5 codes faux**, la remise est bloquée et l'admin alerté ; lui seul peut la débloquer.
- **Perte** : remboursement du transport + indemnité = min(valeur déclarée, plafond/kg × poids) si un plafond est réglé.
- **Incident** (signalé par le transporteur ou l'admin) : l'admin est alerté, le client prévenu. L'admin le traite : *retrouvé* (le suivi reprend), *perdu* (remboursement automatique) ou *résolu*.

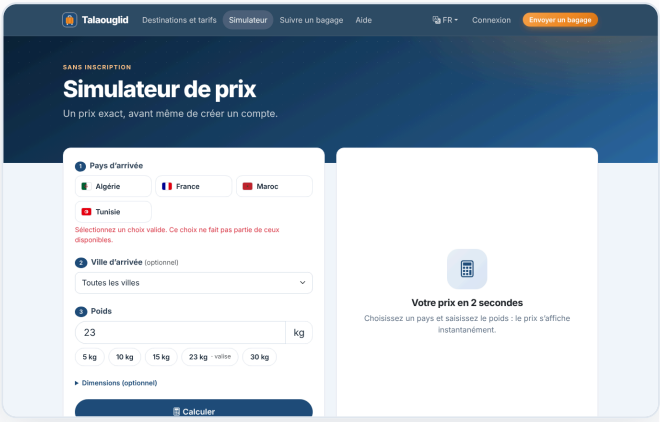
7. Les pages du site (adresses et rôles)

Espace public

Adresse	Page
`/`	Accueil : bandeau avec suivi rapide, chiffres clés, 3 prochains départs, étapes, pays desservis, avis, appel à l'action.
`/destinations/`	Une carte par pays actif : drapeau, villes, prix de départ, frais fixes, tarif/kg, poids max, délai, prochain départ, objets interdits, documents de douane. Filtre par pays.
`/simulateur/`	Calcul instantané du prix (pays, ville, poids, dimensions), décomposition, délai, voyages disponibles.
`/suivi/` et `/suivi/<numéro>/`	Recherche, puis statut, barre de progression, carte du trajet, historique, abonnement par courriel.
`/aide/`	FAQ avec recherche, coordonnées (téléphone, WhatsApp, courriel), formulaire de contact.
`/conditions/`	Conditions générales et politique de confidentialité.
`/connexion/`, `/inscription/`, `/mot-de-passe/…`	Comptes : connexion, création, confirmation du courriel, mot de passe oublié ou changé.
`/notifications/`	Toutes les notifications de l'utilisateur connecté.
`/robots.txt`, `/sitemap.xml`	Pour Google : pages à indexer, espaces privés exclus.
`/2fa/`	Double authentification des administrateurs.
`/avis/<jeton>/`	Formulaire d'avis reçu par le client 24 h après la livraison.
`/taches/verifications/`	Tâches périodiques, protégées par jeton (pour cron-job.org).



Destinations et tarifs



Simulateur de prix

Espace client (`/client/`)

Adresse	Page
`/client/`	Tableau de bord : réservations à payer, envois en cours, historique, notifications.
`/client/envoi/nouveau/` (étapes 1 à 4)	1. Destination et voyage · 2. Bagage · 3. Destinataire · 4. Récapitulatif et réservation.
`/client/envois/<numéro>/`	Détail : encadré « à payer au comptoir », statut, carte, historique, code de remise, reçu PDF, annulation.
`…/recu.pdf` , `…/annuler/` , `…/choix/`	Reçu, annulation, choix après annulation d'un voyage.
`/client/profil/`	Coordonnées, langue, notifications, destinataires enregistrés, suppression du compte.

Espace transporteur (`/transporteur/`) — pensé pour le téléphone

Adresse	Page
`/transporteur/`	Voyage en cours (gros bouton GPS, temps depuis la dernière position), prochains voyages, voyages terminés.
`/transporteur/voyages/<id>/`	Compteur chargés/total, saisie ou scan d'un numéro, démarrer, arrivée, bagages avec ville de livraison, carte, incident.
`/transporteur/position/`	Position GPS en 1 clic, ou saisie manuelle du lieu.
`/transporteur/livraison/<numéro>/`	Destinataire, ville, code à 6 chiffres, photo, validation.

Espace administrateur (`/gestion/`)

Adresse	Page
`/gestion/`	Chiffres du mois, réservations à encaisser, alertes, carte des voyages en cours, derniers paiements et réservations, export Excel.
`/gestion/pays/`	Liste des pays, ajout, modification (villes avec coordonnées), activation, suppression ou désactivation.
`/gestion/voyages/`	Liste ou calendrier mensuel, création, modification, duplication (+7 jours), annulation.
`/gestion/bagages/`	Recherche et filtres (dont « À encaisser »), fiche complète : encaisser, changer le statut, déplacer, rembourser, incident, reçu.
`/gestion/bagages/comptoir/`	Envoi au comptoir pour un client venu sans réservation.
`/gestion/incidents/`	Incidents ouverts et traités.
`/gestion/utilisateurs/`	Clients et transporteurs, création d'un transporteur, blocage, réinitialisation du mot de passe.
`/gestion/messages/` , `/gestion/parametres/`	Messages de contact ; réglages du site (dont informations légales).

`/gestion/avis/`	Modération des avis clients.
`…/bagages/<n°>/etiquette.pdf`	Étiquette avec code QR à coller sur le bagage.

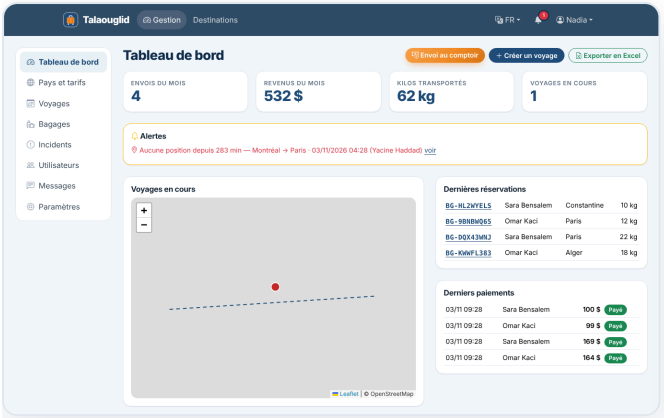
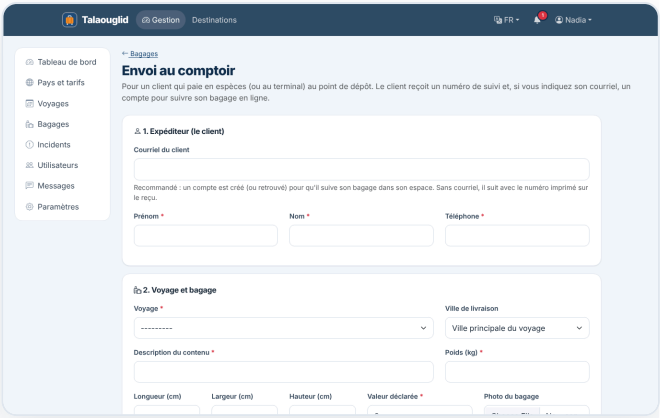


Tableau de bord administrateur



Envoi au comptoir

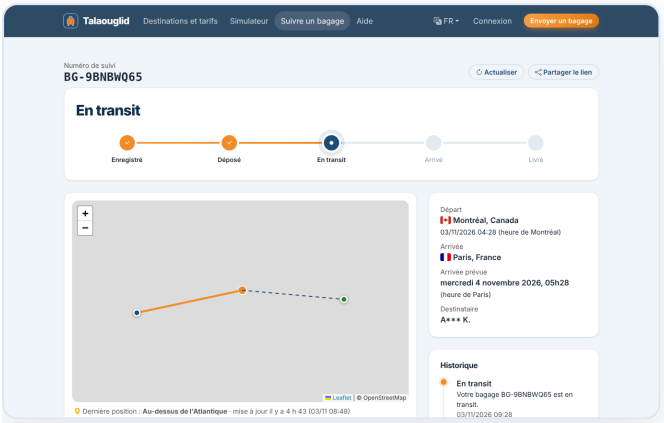
8. Parcours complets (de A à Z)

A. Un client envoie une valise à Lyon

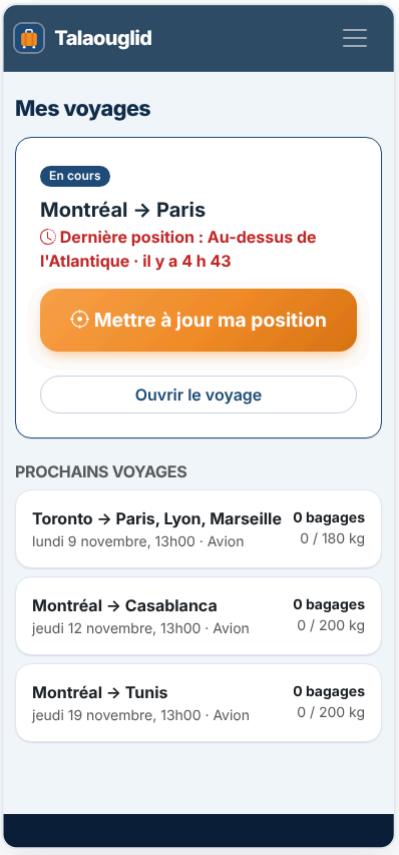
1. Il utilise le **simulateur** : France, 23 kg → prix et voyages disponibles.
2. Il clique sur « Réserver ce prix », crée son compte et confirme son courriel.
3. Étape 1 : il choisit **Lyon** ; il voit les voyages qui desservent Lyon (ex. Toronto → Paris, Lyon, Marseille).
4. Étapes 2 et 3 : bagage (description, poids, photo) et destinataire.
5. Étape 4 : récapitulatif, conditions, « Réserver — paiement au dépôt ». Ses kilos sont bloqués.
6. Il dépose sa valise ; l'admin clique sur **Encaisser**. Le client reçoit son numéro de suivi, son reçu et le code de remise à donner au destinataire.

B. Le transporteur fait le voyage

1. Il ouvre son voyage, saisit ou scanne chaque numéro : « chargé ».
2. « Démarrer le voyage » : bagages en transit, clients notifiés.
3. Il envoie sa position (aéroport…), puis la met à jour régulièrement (rappel toutes les 2 h).
4. « Marquer arrivé » : bagages « arrivés à destination ».
5. À Lyon, le destinataire donne son code : le transporteur le saisit, prend une photo, valide → « livré », le client est notifié.



Page de suivi publique d'un bagage en transit



Espace transporteur (téléphone)

C. Incident : une valise manque

1. Le transporteur signale « bagage introuvable ».
2. L'admin est alerté ; le bagage passe « incident en cours » ; le client est prévenu.
3. L'admin le marque « retrouvé » (le suivi reprend) ou « perdu » (remboursement).

D. Annulation

1. Le client annule avant la date limite : remboursé s'il avait payé, sinon simple annulation ; kilos libérés.
2. L'admin annule un voyage : chaque client choisit un autre voyage ou le remboursement depuis sa page d'envoi.

E. L'admin ajoute un pays et planifie un voyage

1. Gestion → Pays et tarifs → « Ajouter » : nom, code, tarifs, délai, villes (« Nom; latitude; longitude »), objets interdits, documents.
2. Gestion → Voyages → « Créer » : départ, ville principale, autres villes, dates, dépôt, capacité, transporteur. Le transporteur est notifié ; le voyage apparaît sur l'accueil.

9. Notifications et courriels

Chaque notification apparaît dans le site (cloche du menu) et, si le client l'a choisi, par courriel. Les SMS sont prévus pour la version 2.

Événement	Qui est prévenu	Comment
Réservation à payer au comptoir	Client ; administrateurs	Site + courriel (montant, lieu, date limite) ; site pour l'admin
Paiement confirmé	Client ; destinataire	Courriel : numéro, reçu, lieu, date limite, code de remise ; SMS au destinataire (journalisé)
Chargé, en transit, arrivé, livré	Client ; abonnés au suivi	Site + courriel
Rappel de date limite (24 h avant)	Client	Courriel
Rappel de position (2 h)	Transporteur	Site + courriel
Aucune position depuis 4 h	Administrateurs	Site + courriel
Nouveau voyage assigné	Transporteur	Site + courriel
Dates d'un voyage modifiées	Clients du voyage, transporteur	Site + courriel
Voyage annulé	Clients, transporteur	Site + courriel
Incident signalé	Administrateurs ; client	Site + courriel
Voyage presque complet (90 %)	Administrateurs	Site
Nouveau message de contact	Administrateurs	Site + courriel
Compte créé au comptoir	Client	Courriel avec lien pour choisir le mot de passe

En ligne actuellement, aucun courriel ne part : il faut configurer un service d'envoi (Brevo gratuit jusqu'à 300 courriels par jour) avec les variables `EMAIL_HOST`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` sur Render. En attendant, les notifications dans le site fonctionnent.

10. Commandes, tâches planifiées et API

Commandes (à lancer avec `python manage.py …`)

Commande	Ce qu'elle fait
`runserver`	Lance le site sur le PC (http://127.0.0.1:8000).
`migrate`	Met la base à jour (à faire après chaque `git pull` qui contient une migration).
`makemigrations`	Crée une migration après une modification de `models.py`.
`test core`	Lance les 53 tests automatiques.
`donnees_demo [--reinitialiser]`	Crée pays, villes, voyages, comptes et envois d'exemple.
`supprimer_demo`	Supprime les comptes `@demo.ca`, leurs voyages, envois, paiements et les avis de démo ; garde les pays, les tarifs et les vrais comptes.
`creer_admin`	Crée l'admin défini par `DJANGO_ADMIN_EMAIL` / `DJANGO_ADMIN_PASSWORD` s'il n'existe pas (le mot de passe n'est plus écrasé). `DJANGO_ADMIN_RESET=true` force la remise du mot de passe et de la double authentification.
`verifications`	Rappels de position (2 h), alertes sans position (4 h), rappels de date limite (24 h), annulation des réservations non payées expirées, demandes d'avis. En ligne : appelée par cron-job.org via `/taches/verifications/`.
`createsuperuser`	Crée un administrateur sur la base du PC.

API JSON (`/api/`)

Adresse	Contenu	Accès
`/api/suivi/<numéro>/`	Statut, étape, destinataire masqué, départ, arrivée, positions, historique.	Public
`/api/pays/`	Pays actifs avec tarifs et villes.	Public
`/api/voyages-en-cours/`	Voyages en cours, dernière position, minutes depuis la position (carte de l'admin).	Administrateur

Interface d'administration de secours

`/django-admin/` donne un accès brut à toutes les tables (pour l'administrateur). À n'utiliser qu'en cas de besoin : l'espace `/gestion/` applique les règles de gestion, pas celle-ci.

11. Interface : design, langues, accessibilité

Design

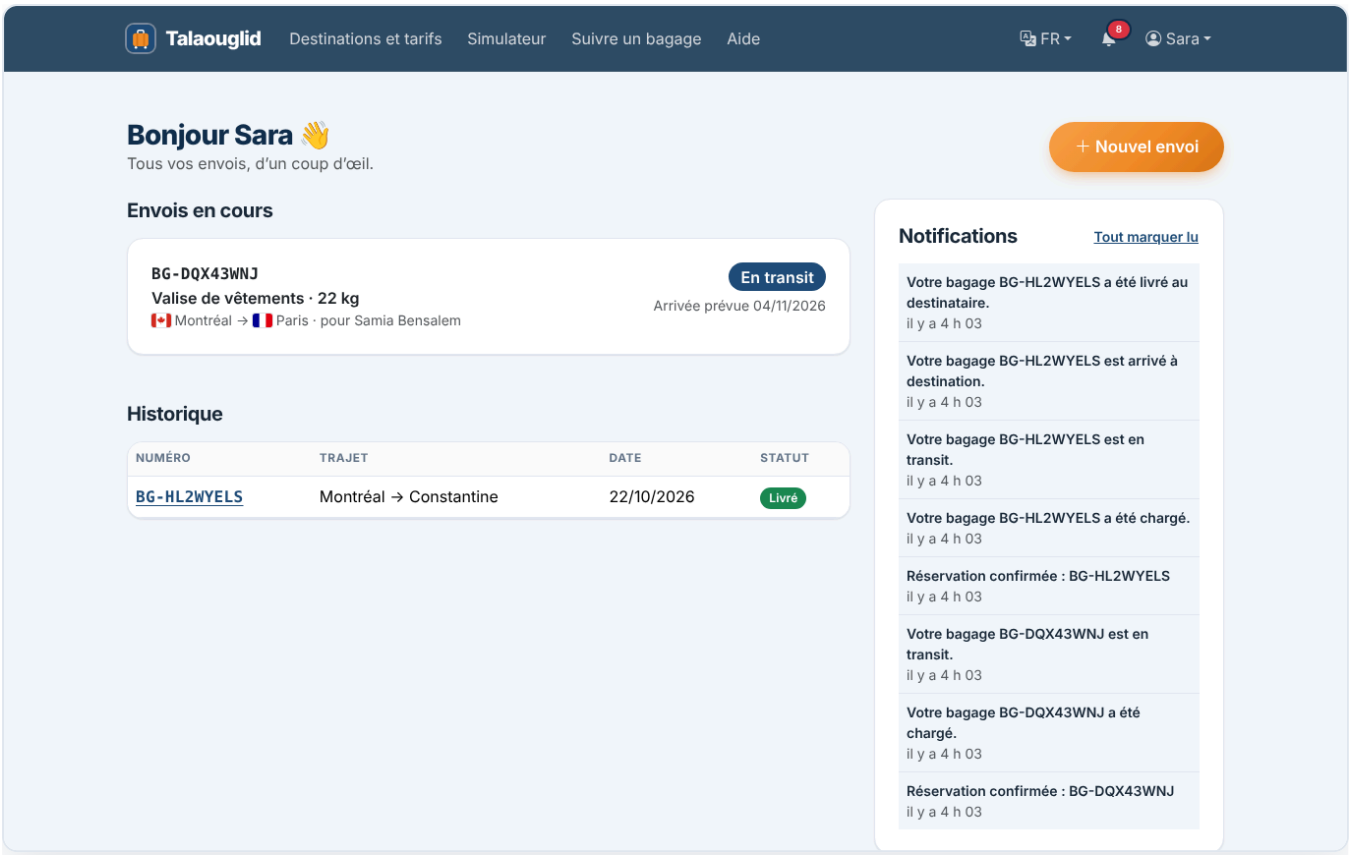
- **Couleurs** : bleu nuit `#1f4e79` et orange `#f28c28` ; fond gris très clair ; coins arrondis.
- **Logo** : valise orange sur fond bleu (onglet, téléphone, menu, pied de page, aperçu des liens partagés).
- **Menu** semi-transparent, qui s'affine au défilement ; la page courante est mise en évidence.
- **En-têtes bleus dégradés** sur toutes les pages publiques ; avion animé sur l'accueil.
- **Messages flottants** en haut à droite, qui disparaissent seuls (sauf les erreurs).
- **Boutons** avec indicateur de chargement après un clic (évite les doubles envois).
- Apparition douce des sections au défilement ; transitions entre les pages (Chrome, Edge).
- Adapté au **téléphone** (espace transporteur pensé d'abord pour le mobile, gros boutons).

Langues

Français par défaut ; anglais et arabe au choix (menu « FR / EN / AR »). En arabe, la page passe de droite à gauche. Les traductions sont dans `locale/`, générées par `scripts/traductions.py`. **Aujourd'hui, environ 135 textes sont traduits** (navigation, accueil, statuts…) ; le reste s'affiche en français.

Accessibilité

- Contrastes suffisants, navigation au clavier (contour orange sur l'élément actif), lien « Aller au contenu ».
- Textes alternatifs, étiquettes de formulaire, zones annoncées aux lecteurs d'écran.
- Si le visiteur a demandé de réduire les animations sur son appareil, elles sont désactivées.



Espace client

12. Sécurité et confidentialité

Domaine	Mesure
Connexion	HTTPS partout ; mots de passe chiffrés ; règles de mot de passe solides ; **blocage 1 h après 5 échecs** (même compte, même adresse IP).
Administrateurs	**Double authentification obligatoire** (code à 6 chiffres d'une application).
Robots	Champ piège sur le contact et l'inscription ; limite de 30 requêtes/min sur le suivi et l'API.
Remise	Code comparé de façon sûre ; blocage après 5 codes faux ; photo obligatoire.
Accès	Rôle vérifié sur chaque page ; un client ne voit que ses envois ; un transporteur que ses voyages.
Formulaires	Protection CSRF sur tous les envois de formulaires.
Numéro de suivi	Aléatoire et non devinable (8 caractères, 32 possibilités chacun).
Page de suivi publique	Nom du destinataire masqué ; code de remise jamais affiché ; **position floutée** (dernier lieu nommé et heure, aucune coordonnée) ; non indexée par Google.
Paiement	Aucune donnée de carte stockée (Stripe, quand il sera activé).
Loi 25 (Québec)	Consentement, politique de confidentialité (hébergement aux États-Unis, responsable désigné), suppression du compte par le client.
Traçabilité	Chaque changement de statut garde la date, l'heure et l'auteur.
Secrets	Clé secrète, mots de passe et clés API dans les variables Render, jamais dans le code ni sur GitHub.

À faire avant de vrais clients : supprimer les comptes de démo, configurer la double authentification de chaque admin, et utiliser un mot de passe unique.

13. Tests automatiques

53 tests dans `core/tests.py`, lancés avec `python manage.py test core`. **Tous réussissent.** Chaque test recrée une base vide, joue un parcours, puis vérifie le résultat.

Groupe	Ce qui est vérifié	Tests
Scénario 1 — Ajout d'un pays	L'admin ajoute la Tunisie (visible partout, prix 15 kg = 147,50 $) ; un pays utilisé est désactivé et non supprimé ; accès réservé à l'admin.	3
Scénario 2 — Planifier un voyage	Création + notification du transporteur ; dates incohérentes refusées ; pays identiques refusés ; date limite par défaut 24 h.	4
Scénario 3 — Réservation	Parcours complet (164 $, numéro, code, capacité 282 → 264 kg, courriel, reçu PDF) ; poids maximum ; voyage complet ; réservations fermées ; prix figé ; carte refusée.	6
Scénario 4 — Transporteur	Chargement, démarrage, position, rappel à 2 h, alerte à 4 h (une seule fois), arrivée ; un autre transporteur n'a pas accès.	2
Scénario 5 — Suivi	Page publique (statut, position, nom masqué, code caché), API ; numéro inconnu ; abonnement par courriel.	3
Scénario 6 — Livraison	Mauvais code refusé, bon code accepté, client notifié.	1
Scénario 7 — Incident	Introuvable → alerte → retrouvé (suivi reprend) ; perdu → remboursement ; historique daté.	1
Scénario 8 — Annulation	Client avant la date limite ; impossible après dépôt ; voyage annulé → autre voyage ou remboursement.	3
Comptes	Inscription, courriel de confirmation, redirection par rôle ; suppression de compte (Loi 25).	2
Envoi au comptoir	Nouveau client (compte + lien), client existant, sans courriel, remboursement en espèces, accès admin.	4
Réservation + paiement au comptoir	Kilos bloqués puis encaissement (pas compté deux fois) ; capacité ; annulation et expiration libèrent les kilos.	3
Suppression de la démo	Comptes, voyages et envois de démo supprimés ; vrais comptes et pays conservés.	1
Voyages vers plusieurs villes	Escales du même pays seulement ; réservation pour Oran ; filtrage par ville ; ville non desservie refusée.	3
Tâches planifiées	Jeton obligatoire ; `creer_admin` n'écrase plus le mot de passe.	3
Comptoir et remise	Pesée → prix recalculé ; pesée hors capacité refusée ; poids volumétrique ; blocage après 5 codes faux puis déblocage.	4
Sécurité	Double authentification admin ; blocage après 5 échecs ; piège à robots ; position publique floutée.	4
Mise en règle	NEQ et nom légal affichés, mention de l'hébergement, indemnité plafonnée, « $ CA ».	1

Villes et langues	Doublons refusés ; menu des langues masqué ; pas de Google Fonts.	2
Après lancement	Étiquette QR, WhatsApp, demande d'avis à 24 h + modération, création d'un admin.	3

14. Hébergement et mise en ligne (Render)

Ce qui tourne sur Render

Élément	Détail
Service web `talaouglid`	Python, plan gratuit, Oregon. Adresse : https://talaouglid.onrender.com. Mise en ligne automatique à chaque envoi sur `main`.
Base `talaouglid-db`	PostgreSQL 18, plan gratuit, Oregon, 1 Go. **Expire le 2 décembre 2026** si elle n'est pas passée au plan payant.
Blueprint `Talaouglid.com`	Relie le dépôt GitHub au fichier `render.yaml`.

> **Important** : le site et la base doivent être dans la **même région**, sinon le site ne trouve pas la base (« failed to resolve host »). C'était la cause des premiers échecs de déploiement.

Ce qui se passe à chaque mise en ligne

1. **Construction** (`build.sh`) : installation des paquets, préparation des fichiers statiques. La base n'est pas accessible à ce moment-là.
2. **Démarrage** (`start.sh`) : attente de la base (jusqu'à 2 min), migrations, création de l'admin, suppression ou chargement de la démo selon les variables, lancement de Gunicorn.
3. Le statut passe à **Live**. En cas d'échec, l'ancienne version reste en ligne ; « Rollback » permet de revenir à une version précédente.

Variables d'environnement (Render → talaouglid → Environment)

Variable	Rôle
`DATABASE_URL`	Adresse de la base (remplie par Render — **ne pas toucher**).
`DJANGO_SECRET_KEY`	Clé secrète (générée par Render — **ne pas toucher**).
`DJANGO_DEBUG`	`false` en ligne.
`DJANGO_ADMIN_EMAIL` / `DJANGO_ADMIN_PASSWORD`	Votre compte administrateur (recréé ou remis à jour à chaque démarrage).
`CHARGER_DEMO`	`true` = données de démo au démarrage ; `false` en production.
`SUPPRIMER_DEMO`	`true` = supprime la démo au démarrage.
`SERVE_MEDIA`	`true` = Django sert les photos (pas de Nginx sur Render).
`PYTHON_VERSION`	3.12.7.
`STRIPE_SECRET_KEY` (+ `STRIPE_WEBHOOK_SECRET`)	Active le paiement par carte en ligne.

`EMAIL_HOST`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`	Active l'envoi de vrais courriels.
`PAIEMENT_EN_LIGNE`	Facultatif : force le paiement en ligne (`true` / `false`).
`VERIFICATIONS_JETON`	Jeton secret des tâches planifiées (généré par Render).
`DJANGO_ADMIN_RESET`, `DJANGO_ADMIN_2FA`	Urgence : remettre le mot de passe et la double authentification ; désactiver temporairement la double authentification.
`S3_BUCKET`, `S3_ENDPOINT_URL`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`	Stockage des photos (Cloudflare R2).
`DJANGO_DOMAINE`, `SENTRY_DSN`, `DJANGO_LANGUES`, `DEFAULT_FROM_EMAIL`	Nom de domaine ; alertes d'erreurs ; langues proposées ; expéditeur des courriels.

Limites du plan gratuit

- Le site s'endort après 15 min sans visite ; la visite suivante attend 30 à 60 s.
- La base expire après 30 jours (2 décembre 2026).
- Sans stockage R2, les photos sont effacées à chaque mise à jour.

Le plan payant **Starter** (environ 7 $ US/mois pour le site, autant pour la base) lève ces limites. Pour un serveur personnel (OVHcloud), le dossier `deploiement/` contient tout le nécessaire.

15. Travailler sur le projet au quotidien

Installer le projet sur un PC (Windows, PyCharm)

```
git clone https://github.com/achour2/transport-am.git
cd transport-am
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py donnees_demo
.venv\Scripts\python manage.py runserver        →  http://127.0.0.1:8000
```

Dans PyCharm : `<No interpreter>` → *Add Local Interpreter* → *Select existing* → `.venv\Scripts\python.exe`. Toujours utiliser `.venv\Scripts\python` (sinon : « No module named 'django' »).

Faire une mise à jour

```
git pull origin main                      # 1. récupérer la dernière version
(modifier le code dans PyCharm)           # 2. travailler
.venv\Scripts\python manage.py makemigrations   # seulement si models.py a changé
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py test core        # 3. vérifier
git add -A
git commit -m "Ce que j'ai changé"
git push origin main                      # 4. Render met en ligne tout seul
```

> **Votre PC et le site en ligne ont deux bases séparées.** Un compte ou un voyage créé sur le PC n'existe pas en ligne : le push n'envoie que le code. Les données en ligne se gèrent dans l'espace `/gestion/` du site en ligne, et les réglages secrets dans Render → Environment.

Où modifier quoi ?

Je veux changer…	Où
Nom, téléphone, adresse, horaires, conditions	Site en ligne → Gestion → Paramètres (sans code).
Pays, tarifs, villes, objets interdits	Gestion → Pays et tarifs (sans code).
Couleurs, tailles, espacements	`static/css/site.css` (variables au début du fichier).
Texte ou mise en page d'une page	Le gabarit correspondant dans `templates/` (section 4).
Une règle (prix, capacité, annulation…)	`core/services.py` (et `core/models.py` pour le prix).
Un champ de formulaire	`core/forms.py`.
Une nouvelle information à enregistrer	`core/models.py` puis `makemigrations`.
Les questions de la FAQ	Liste `FAQ` dans `core/views/public.py`.

Les traductions	`scripts/traductions.py` puis l'exécuter.
L'hébergement	`render.yaml`, `build.sh`, `start.sh`.

16. Limites actuelles et prochaines étapes

Indispensable avant de vrais clients

Action	Pourquoi
Supprimer les données de démo	Mot de passe connu. `CHARGER_DEMO=false` + `SUPPRIMER_DEMO=true`.
Saisir les vraies données	Paramètres, pays, tarifs, villes, transporteurs, voyages. Corriger les données en double (ex. « tizi ouzou » et « Tizi-Ouzou ») et les devises.
Brancher les courriels (Brevo), les rappels (cron-job.org), les photos (R2)	Le code est prêt : voir le « Guide des actions de votre côté ».
Base payante avant le 2 décembre 2026	Sinon toutes les données sont supprimées.

Améliorations utiles

- Traduction complète en anglais et en arabe (menu masqué en attendant).
- Nom de domaine `talaouglid.com` + Google Search Console.
- Paiement par carte en ligne (Stripe).

Version 2 (cahier des charges)

- SMS / WhatsApp au destinataire avec le code de remise.
- Suivi GPS automatique en continu ; application mobile transporteur.
- Codes promo, fidélité ; assurance optionnelle.
- Statistiques avancées, exports comptables.

Points encore à valider

- Prix au volume (gros cartons légers) ; tarifs différents par ville.
- Devise de facturation (CAD ou devise du pays).
- Ramassage à domicile ; politique de remboursement en cas de perte.

17. Historique des versions

Commit	Date	Changement
`263e16f`	02/11/2026	Première version complète selon le cahier des charges v1.0 (4 espaces, 8 scénarios testés).
`1d6eb8c`	02/11/2026	Envoi au comptoir : l'admin enregistre un client qui paie en espèces.
`57f708e`	03/11/2026	Préparation de Render (WhiteNoise, `DATABASE_URL`, `render.yaml`).
`d7cf18e`	03/11/2026	Migrations au démarrage (base inaccessible pendant la construction).
`9d6d1da`	03/11/2026	Nouveau nom Talaouglid + référencement (titres, description, robots.txt, sitemap.xml).
`913f555`	03/11/2026	Service et base renommés `talaouglid`.
`ebea473`	03/11/2026	Le démarrage attend que la base soit prête.
`83322a4`	03/11/2026	Site et base dans la même région (Oregon) — site en ligne.
`a291f97` / `4fbbe2b`	03/11/2026	Suppression des données de démo ; plus de démo par défaut.
`11a252c`	03/11/2026	Icône du site (valise).
`f28b256`	03/11/2026	Paiement en ligne désactivé : réservation en ligne, encaissement au comptoir.
`345b0ab`	03/11/2026	Interface modernisée (accueil, animations, menu, messages flottants).
`1ec39d8`	03/11/2026	Pages Destinations, Simulateur, Suivi, Aide refaites ; vrais drapeaux.
`cf51813`	03/11/2026	Voyages vers plusieurs villes du même pays + ville de livraison par envoi.
`ff2ee35`	03/11/2026	Plan S1 : tâches planifiées par jeton, Celery retiré, mot de passe admin conservé, sauvegarde PC.
`fbb7b3e`	03/11/2026	Plan S2 : pesée au comptoir, poids volumétrique, photo de remise obligatoire, 5 essais de code, photos R2.
`ddb0442`	03/11/2026	Plan S2 : blocage des connexions, piège à robots, limite de requêtes, position floutée, double authentification.
`f98a804`	03/11/2026	Plan S3 : informations légales, conditions et confidentialité complètes, déclaration de contenu, indemnisation, $ CA.
`54dd694`	03/11/2026	Plan S4 : villes sans doublons, langues masquées, polices locales, Sentry, tests GitHub Actions.
`f92ce2f`	03/11/2026	Étiquette QR, WhatsApp, demandes d'avis, création d'administrateurs.

18. Aide-mémoire

Liens

Site : https://talaouglid.onrender.com
Gestion : .../gestion/
GitHub : github.com/achour2/transport-am
Render : dashboard.render.com

Formule du prix

frais fixes + poids × tarif/kg
18 kg → Algérie = 20 + 18 × 8 = 164 $

Cycle d'un bagage

Enregistré → Déposé → Chargé → En transit → Arrivé → Livré

ou Incident / Perdu / Annulé

Cycle d'un voyage

Planifié → En cours → Terminé

ou Annulé (clients notifiés)

Délais par défaut

Fermeture des réservations : 24 h avant le départ
Rappel de position : 2 h · Alerte admin : 4 h
Alerte « presque complet » : 90 %

En cas de problème

Render → talaouglid → Logs (dernières lignes)
Version cassée → Deploys → Rollback
« No module named 'django' » → utiliser `.venv\Scripts\python`

Commandes les plus utiles

```
.venv\Scripts\python manage.py runserver      # lancer le site sur le PC
.venv\Scripts\python manage.py migrate        # mettre la base à jour
.venv\Scripts\python manage.py test core      # lancer les tests
git pull origin main                          # récupérer la dernière version
git add -A ; git commit -m "..." ; git push origin main   # mettre en ligne
```